Portfolio Management System - Requirements & Design Specification

1. System Overview

A Streamlit-based retirement portfolio management system for tracking investments across multiple accounts, institutions, and asset classes. The system implements a three-bucket investment strategy (Growth Funds, Bond Ladder, Cash Reserve) with comprehensive transaction tracking and historical price management.

2. Functional Requirements

2.1 Core Functionality

- **Multi-account Management**: Track investments across different institutions (brokers, banks)
- **Transaction Recording**: Manual entry and CSV import of all transaction types
- **Historical Price Storage**: Store and update daily OHLCV data from yfinance
- **Multi-currency Support**: Handle GBP, GBp, USD, EUR with broker exchange rates
- **Investment Strategy Tracking**: Three-bucket allocation (Growth Funds, Bond Ladder, Cash Reserve)

2.2 Security Types Supported

- **Equity Funds**: ETFs, mutual funds, index funds
- **Government Bonds (Gilts)**: Individual UK government bonds with maturity tracking
- **Money Market Funds**: Cash-equivalent investments
- **Cash Holdings**: Bank account balances

2.3 Transaction Types

- **Trades**: Buy/sell securities with fees
- **Income**: Dividends, bond coupons, interest payments
- **Fees**: Platform fees, custody charges
- **Transfers**: Between accounts including external sources (bequests, property sales)

2.4 Data Import/Export

- **CSV Import**: Flexible format for bulk transaction entry
- **CSV Export**: Backup capability for all data tables
- **Historical Data**: Automatic yfinance integration for price updates

3. Technical Requirements

3.1 Technology Stack

- **Frontend**: Streamlit web application
- **Database**: SQLite with SQLAlchemy ORM
- **Data Source**: yfinance for historical prices
- **Language**: Python 3.8+

3.2 Dependencies

```
streamlit
pandas
sqlalchemy
yfinance
pydantic (data validation)
python-dotenv (configuration)
streamlit-aggrid (enhanced tables)
openpyxl (Excel support)
```

3.3 Performance Requirements

- **Database**: SQLite suitable for personal use
- **Price Updates**: On-demand refresh (not real-time)
- **Response Time**: < 2 seconds for typical operations
- **Data Volume**: Support for 10+ years of daily price data

3.4 Configuration Requirements

- **Database Path**: User-configurable location
- **Date Format**: dd/mm/yyyy display format
- **Currency Settings**: Default currency preferences
- **Update Frequency**: Configurable yfinance refresh intervals

4. Database Design

4.1 Entity Relationship Model

The system implements a normalized relational database with the following key entities:

Core Entities

- **Accounts**: Financial accounts across institutions
- **Securities**: Stocks, bonds, funds with metadata
- **Transactions**: Central transaction log

- **SecurityPrices**: Historical OHLCV price data

Lookup Tables

- **Owner**: Account ownership (Sam, Barbara, Joint)
- **Institution**: Financial institutions
- **AccountType**: Account categories (SIPP, ISA, Trading)
- **TransactionType**: Transaction categories
- **Bucket**: Investment strategy allocations
- **Currencies**: Multi-currency support

Transaction Detail Tables

- **Trades**: Buy/sell transaction specifics
- **Income**: Dividend/coupon details
- **Fees**: Fee transaction details
- **Transfers**: Account transfer specifics

4.2 Key Database Features

- **Referential Integrity**: Foreign key constraints
- **Multi-currency**: Currency and exchange rate tracking
- **Audit Trail**: Complete transaction history
- **Flexibility**: Support for external accounts and sources

5. Application Architecture

5.1 File Structure

```
portfolio_manager/
├── main.py              # Streamlit entry point
├── config.py            # Configuration management
├── requirements.txt       # Dependencies
│
├── database/
│   ├── __init__.py
│   ├── models.py          # SQLAlchemy models
│   ├── database.py        # Database connection & setup
│   └── sample_data.py       # Optional seed data
│
├── services/
│   ├── __init__.py
│   ├── yfinance_service.py  # yfinance API integration
│   ├── portfolio_service.py # Business logic
│   └── import_service.py    # CSV import/export
│
├── pages/
│   ├── __init__.py
│   ├── dashboard.py       # Landing page (future)
│   ├── portfolio.py       # Main page with tabs
│   ├── reports.py         # Analysis (future)
│   └── data_import.py       # Bulk import (future)
│
├── components/
│   ├── __init__.py
│   ├── sidebar.py         # Navigation
│   └── common.py          # Reusable components
│
└── utils/
    ├── __init__.py
    ├── helpers.py         # Utility functions
    └── validators.py      # Data validation
```

5.2 Page Structure

- **Dashboard**: Landing page with portfolio overview
- **Portfolio**: Main working page with 4 tabs:
  - Securities (Funds/Bonds sub-tabs)
  - Transactions (entry and viewing)
  - Accounts (management and CSV backup)
  - Configuration (system settings)
- **Reports**: Analysis and performance tracking

- **Data Import**: Bulk CSV import functionality

5.3 Design Patterns

- **Service Layer**: Business logic separation
- **Repository Pattern**: Data access abstraction
- **Configuration Management**: Environment-based settings
- **Error Handling**: Graceful failure with user warnings

6. Implementation Phases

Phase 1: Foundation (Current)

- ✅ Database schema design and ERD
- ✅ File structure and architecture
- 🔄 Portfolio page with Accounts tab
- 🔄 Basic CRUD operations for accounts
- 🔄 CSV import/export for accounts

Phase 2: Core Functionality

- Securities management (Funds/Bonds tabs)
- Transaction entry and management
- Basic yfinance integration
- Configuration management

Phase 3: Advanced Features

- Dashboard with portfolio overview
- Reports and analytics
- Bulk data import
- Performance optimization

Phase 4: Enhancements

- Advanced bond calculations
- Portfolio rebalancing tools
- Tax reporting features
- Data visualization

7. Data Formats

7.1 Accounts CSV Format

```csv
AccountName,AccountNumber,OwnerName,InstitutionName,AccountTypeName
"My SIPP","ACC123","Sam","Vanguard","SIPP"
"Joint ISA","ISA456","Joint","Hargreaves Lansdown","ISA"
```

7.2 Transactions CSV Format

```csv
Date,Description,AccountName,TransactionType,SecurityTicker,Units,Price,Amount,Currency,ExchangeRate,BucketName
"15/01/2024","Buy VUSA","My SIPP","Buy","VUSA.L","100","75.50","7550.00","GBP","1.0","Growth_Funds"
"15/01/2024","Dividend","My SIPP","Dividend","VUSA.L","","","125.50","GBP","1.0","Growth_Funds"
"15/01/2024","Platform Fee","My SIPP","Fee","","","","25.00","GBP","1.0",""
```

8. Security & Data Integrity

8.1 Data Validation

- Pydantic models for input validation
- Foreign key constraints in database
- Business rule validation in services layer

8.2 Error Handling

- Graceful handling of yfinance API failures
- Warning messages instead of application crashes
- Validation errors with clear user feedback

8.3 Data Backup

- CSV export functionality for all tables
- Configurable database location
- Manual backup procedures

9. User Interface Requirements

9.1 Navigation

- Sidebar with main page navigation
- Tab-based interface within pages
- Consistent styling and layout

9.2 Data Entry

- Form-based manual entry
- CSV upload with validation
- Inline table editing where appropriate

9.3 Data Display

- Tabular data with sorting/filtering
- Multi-currency display (original + GBP)
- Date formatting (dd/mm/yyyy)

10. Future Considerations

10.1 Scalability

- Migration path from SQLite to PostgreSQL
- API-based architecture for mobile apps
- Multi-user support

10.2 Integration

- Direct broker API connections
- Tax software integration
- Financial planning tools

10.3 Analytics

- Performance attribution analysis
- Risk metrics calculation
- Rebalancing recommendations

Implementation Priority: Phase 1 - Accounts Tab

The initial implementation will focus on:

1. Complete file structure setup
2. Database models and connection
3. Portfolio page with stubbed tabs
4. Fully functional Accounts tab with:
    - View/add/edit/delete accounts
    - CSV upload/download

- Validation and error handling

5. Configuration management for database path

This foundation will support incremental development of remaining features.